

Deep learning w chmurze

Wykład 2

Urszula Grzybowska p. 3/95

Algorytm propagacji wstecznej

Wykład 2

PyTorch Workflow

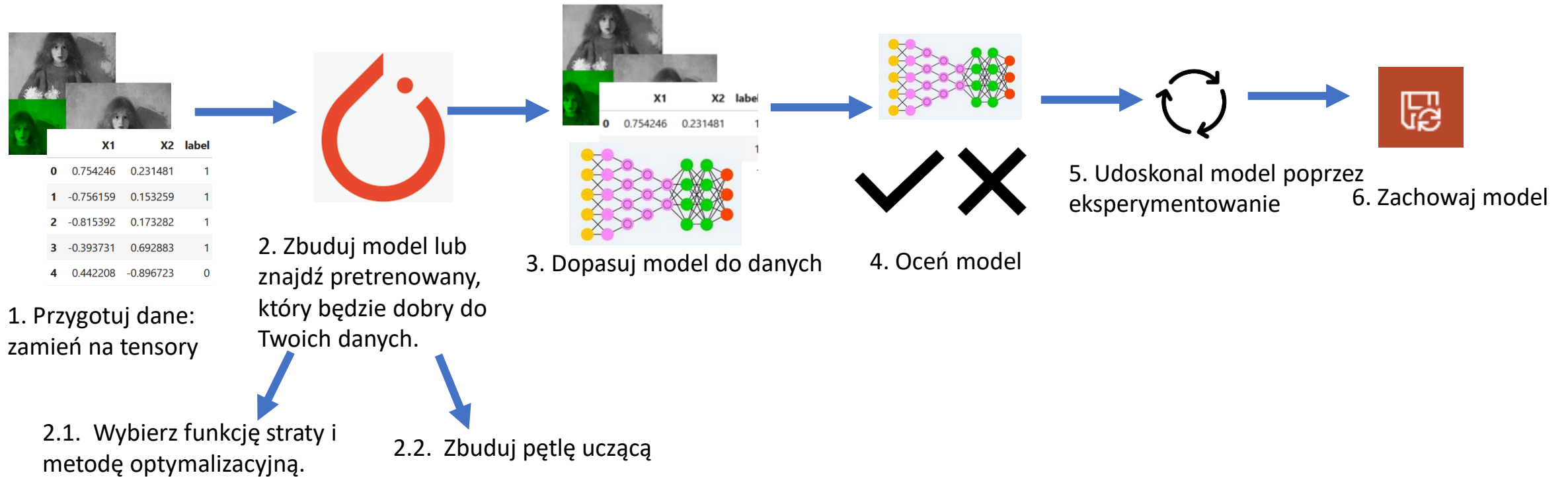


Diagram uczenia na podstawie oznaczonych danych wejściowych

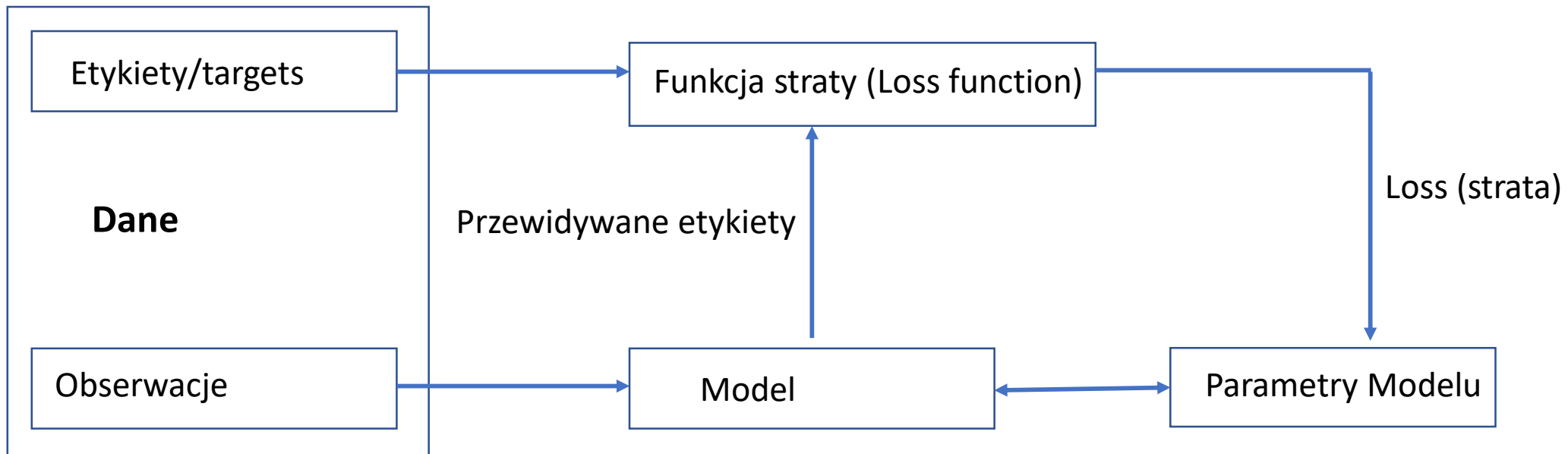
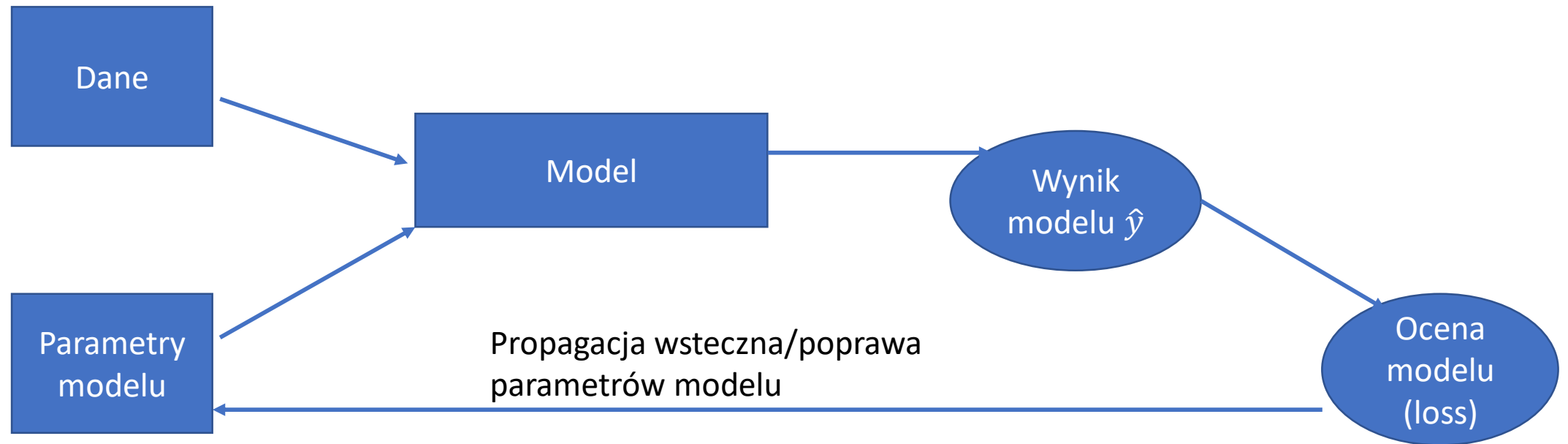


Diagram uczenia



Ostatnia warstwa/warstwa wyjściowa

Rodzaj problemu	Funkcja aktywacji ostatniej warstwy	Funkcja straty
Klasyfikacja binarna	sigmoid	nn.BCELoss (LogLoss)
Wieloklasowa klasyfikacja jednoetykietowa	softmax	nn.CrossEntropyLoss
Wieloklasowa klasyfikacja wieloetykietowa	sigmoid	nn.BCELoss
Regresja wartości rzeczywistych	brak	nn.MSELoss
Regresja wartości z przedziału [0,1]	sigmoid	nn.MSELoss lub nn.BCELoss

[nn.BCEWithLogitsLoss](#) – Połączenie warstwy *Sigmoidalnej* z *BCELoss*.

Optymalizacja w sieci

Parametry modelu sieci neuronowej, wagi w_i oraz b są wyznaczone tak, aby minimalizować odległość między wynikiem sieci a tym, co chcemy uzyskać, czyli etykietą klasy dla instancji danych.

Odległość/błąd mierzą funkcje straty/kosztu.

Liczba neuronów ostatniej warstwy:

- 1 dla regresji i klasyfikacji binarnej
- równa liczbie klas w klasyfikacji wieloklasowej

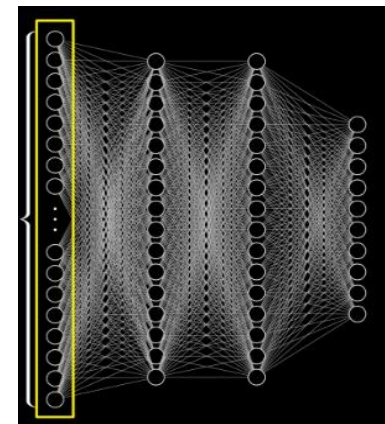
Przykłady w Excelu folder Wykład 2 (ObliczeniaLossW2)

Przykład

Klasyfikacja cyfr pisanych ręcznie. Warstwa wejściowa ma 784 neurony.

Warstwa wyjściowa ma 10 neuronów (po jednym dla każdej cyfry). Dla sieci gęstej z 2 warstwami ukrytymi, każda 16 neuronów, mamy razem

$784*16+16*16+16*10+16+16+10=13002$ parametry (wagi w_{ij} i obciążenia b_j)



An example of hyper-parameters used for training GPT models

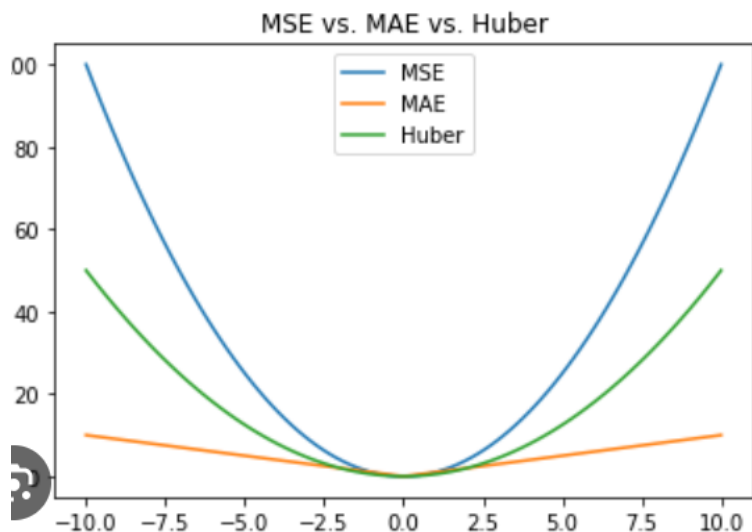
Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Funkcje straty/kosztu (loss functions)

Zadania regresji:

- MSE
- MAE

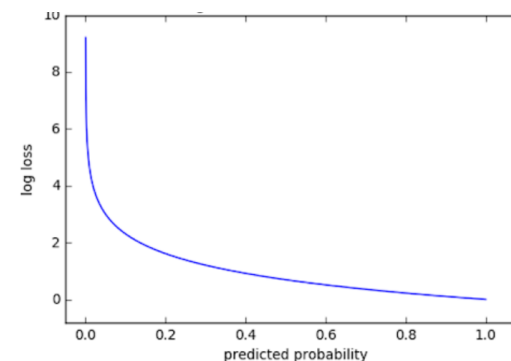
$$L_{MSE}(\hat{y}, y) = \frac{1}{n} \sum (y - \hat{y})^2$$



Zadania klasyfikacji

- Categorical Cross Entropy
- Binary Cross-Entropy-log-loss

$$L_{\log}(y, p) = -(y \log(p) + (1 - y) \log(1 - p))$$



BCELoss

Obliczają błąd naszego modelu, albo inaczej odległość między wynikiem modelu a prawdziwą wartością/etykietą.

Przykłady

Funkcje straty dla j-tego neuronu ostatniej warstwy:

mse (mean squared error) $C(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 = \frac{1}{2n} \sum_{i=1}^n \left(y_j - \Phi(s_j^{(L)}) \right)^2$

gdzie y_i wartość obserwowana, $\hat{y}_i$ wartość przewidywana dla i-tej próbki, n liczba obserwacji w zbiorze treningowym.

entropia krzyżowa (cross entropy) dla klasyfikacji binarnej BCE (logloss)

Dla neuronu j

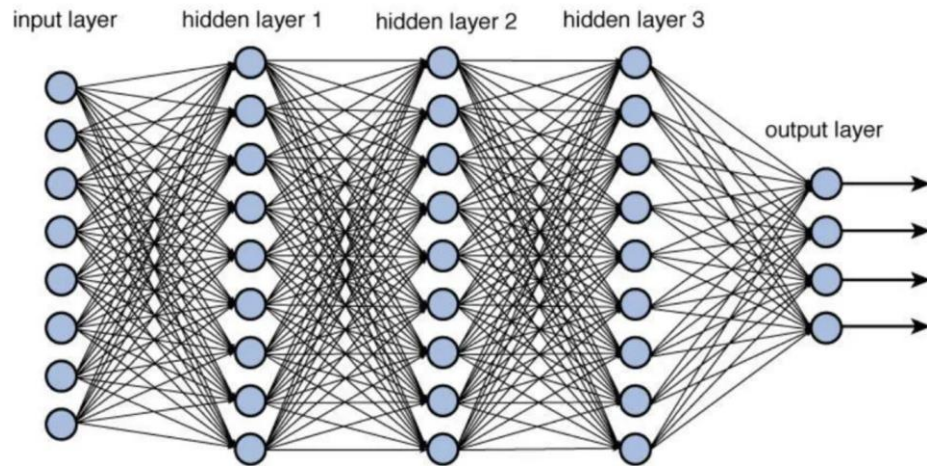
$$C = -\frac{1}{n} \sum_{i=1}^n [y \ln \Phi(s_j) + (1 - y) \ln(1 - \Phi(s_j))],$$

$y \in \{0,1\}$ wartość obserwowana, n liczba obserwacji w zbiorze treningowym.

Głębokie uczenie, w swojej najprostszej formie, może być przedstawione jako klasa funkcji złożonych

$$\left\{ f(x; \theta) = W_L \sigma_L \left(W_{L-1} \dots \sigma_2 \left(W_2 \sigma_1 (W_1 x) \right) \right) \mid \theta = \{W_1, \dots, W_L\} \right\}$$

gdzie $1 \leq l \leq L$ σ_l są pewnymi funkcjami nieliniowymi a $\theta = \{W_1, \dots, W_L\}$ składa się z macierzy o odpowiednich rozmiarach.



$$C(\theta) = g(y, f(x; \theta))$$

fixed

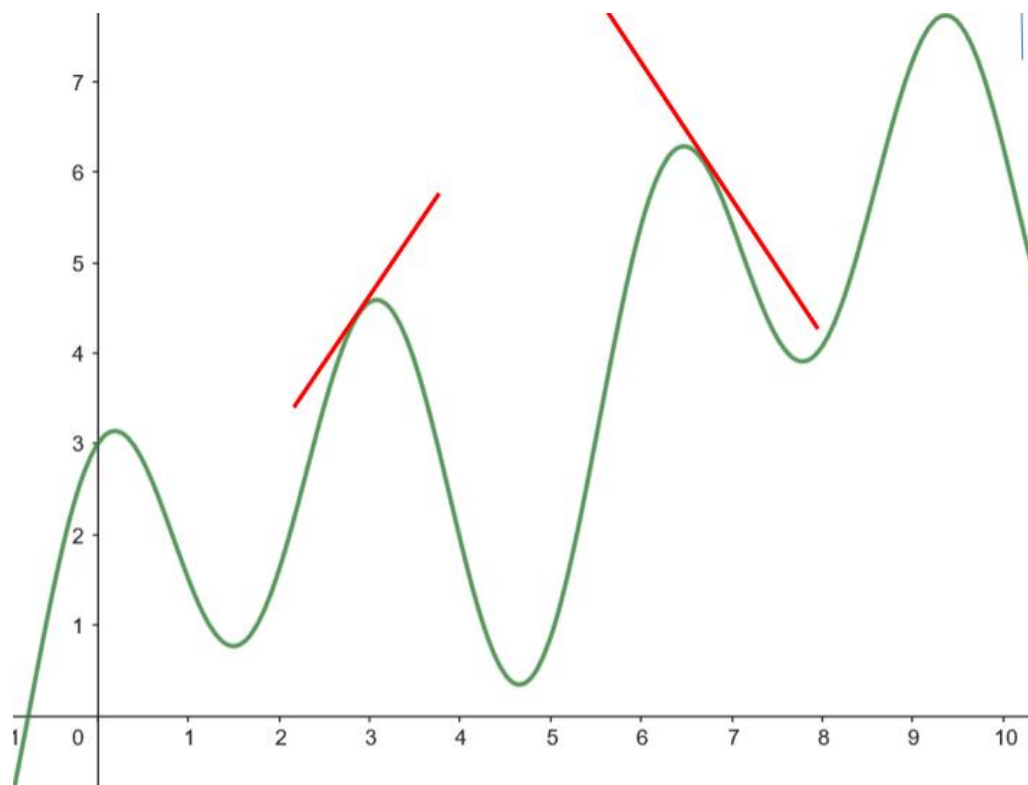
Jest funkcją wag i obciążeń

Cel: Minimalizacja funkcji $C(\theta)$

Środek: obliczanie pochodnych cząstkowych funkcji $C(\theta)$ względem wag i obciążeń.

Geneza-trochę o pochodnych

Pochodna funkcji i jej zastosowanie



Pochodna funkcji różniczkowalnej w punkcie jest współczynnikiem kierunkowym stycznej do wykresu funkcji w tym punkcie. Stąd wskazuje czy wartości funkcji rosną czy maleją (znak pochodnej), czy zmiana jest wolna, czy szybka (wartość bezwzględna pochodnej).

[Animacja](#)

$$y = f'(x_0)(x - x_0) + y_0 \quad y_0 = f(x_0)$$

Równanie stycznej do wykresu funkcji różniczkowalnej

Podstawowe wzory

$$(a)' = 0, \quad a \in R$$

$$(x^\alpha)' = \alpha x^{\alpha-1}, \text{ dla } \alpha \in R, \quad x > 0.$$

$$(e^x)' = e^x$$

$$(\cos x)' = -\sin x \quad (\sin x)' = \cos x$$

$$(\ln x)' = \frac{1}{x}, \quad x > 0$$

$$(a^x)' = a^x \ln a, \quad a \in R^+$$

$$(\log_a x)' = \frac{1}{x \ln a}, \quad a > 0, a \neq 1, \quad x > 0$$

Podstawowe wzory

$$\begin{array}{l} (cf(x))' = cf'(x), \\ (f(x) + g(x))' = f'(x) + g'(x) \\ (f(x) \cdot g(x))' \\ = f'(x) \cdot g(x) + f(x) \cdot g'(x) \end{array} \quad \left[\frac{f(x)}{g(x)} \right]' = \frac{f'(x)g(x) - g'(x)f(x)}{[g(x)]^2}, \quad g(x) \neq 0$$

Reguła łańcuchowa - Chain rule:

$$[f(g(x))]' = f'(g(x)) \cdot g'(x)$$

$$\left\{ f(x; \theta) = W_L \sigma_L \left(W_{L-1} \dots \sigma_2 \left(W_2 \sigma_1 (W_1 x) \right) \right) \mid \theta = \{W_1, \dots, W_L\} \right\}$$

$$\mathcal{C}(\theta) = g(y, f(x; \theta))$$

Jak obliczyć $\frac{\partial \mathcal{C}(\theta)}{\partial w_{ij}}$ i po co?

Gradient funkcji i jego zastosowanie

Wektor pochodnych cząstkowych funkcji f w punkcie x_0 nazywamy gradientem funkcji w x_0

$$\text{Grad}f(x_0) = \nabla f(x_0) = \left[\frac{\partial f}{\partial x_1}(x_0), \frac{\partial f}{\partial x_2}(x_0) \right]$$

Gradient funkcji

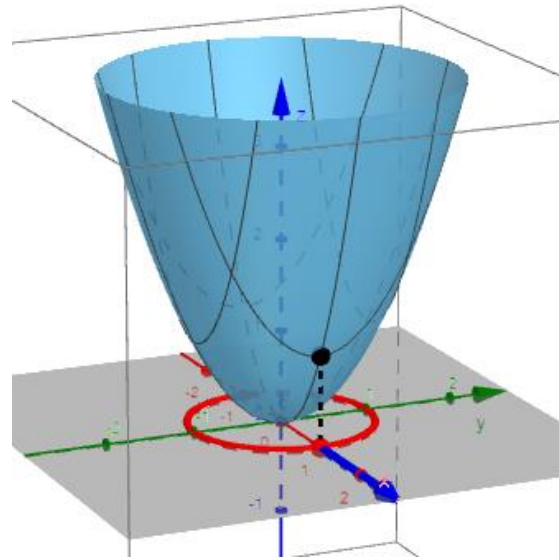
$$\text{Grad} f(x_0) = \nabla f(x_0) = \left[\frac{\partial f}{\partial x_1}(x_0), \frac{\partial f}{\partial x_2}(x_0) \right]$$

$$f(x_1, x_2) = 3x_1x_2^2 + 2x_1x_2 + 3x_1$$

$$\text{Grad} f(x) = [3x_2^2 + 2x_2 + 3, 6x_1x_2 + 2x_1]$$

Gradient funkcji

Animacja



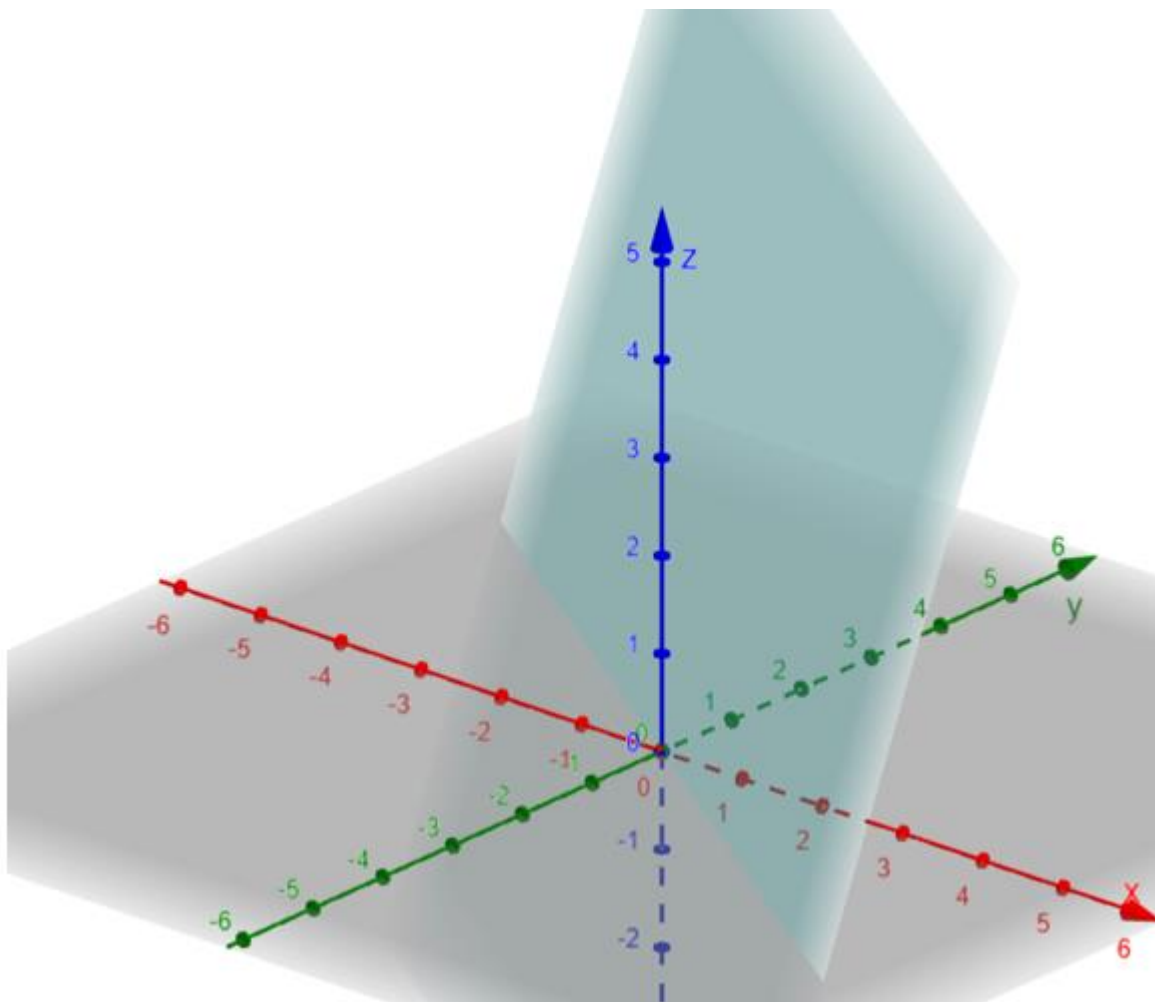
$$f(x_1, x_2) = x_1^2 + x_2^2$$

$$\text{Grad}f = [2x_1, 2x_2]$$

$$\text{Grad}f(1,0) = [2,0]$$

Gradient funkcji oszacowany w punkcie (x_1, x_2) jest normalny do stycznej do warstwicy funkcji przechodzącej przez (x_1, x_2) .
Gradient funkcji oszacowany w punkcie (x_1, x_2) wskazuje kierunek najszybszego wzrostu wartości funkcji.

Gradient funkcji



$$f(x, y) = 2x + 3y$$

$$\nabla f(x, y) = [2, 3]$$

Warstwica funkcji na poziomie a :

$$\{(x, y): f(x, y) = a\}$$

$$f(x, y) = 2x + 3y = 12$$

Gradient funkcji w punkcie jest prostopadły do warstwicy funkcji w tym punkcie i wyznacza kierunek najszybszego wzrostu wartości funkcji.

$$\text{Niech } (x, y) = (3, 2)$$

$$f(3, 2) = 12$$

$$\nabla f(x, y) = [2, 3]$$

Unormujemy gradient $|\nabla| = \sqrt{2^2 + 3^2} = \sqrt{4 + 9} = \sqrt{13}$

Mamy wektor unormowany $[2/\sqrt{13}, 3/\sqrt{13}]$.

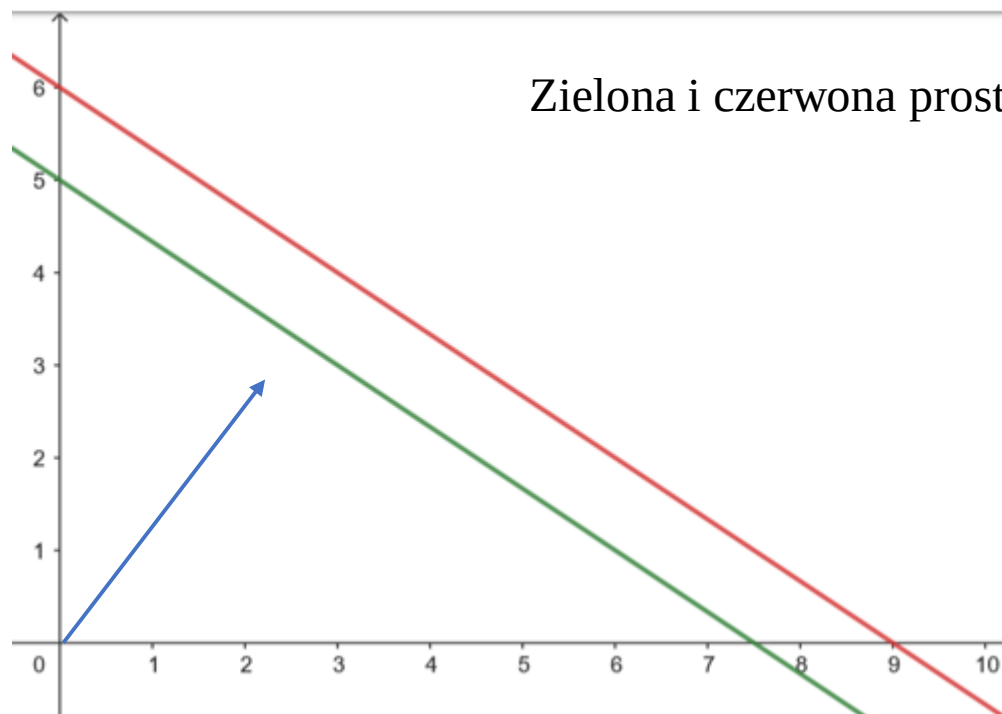
Zobaczmy jak zmieni się wartość funkcji po dodaniu wektora do naszego punktu danych (3,2).
Zaczepiamy wektor, albo myślimy o dodawaniu wektorów $[3,2] + [2/\sqrt{13}, 3/\sqrt{13}]$. Mamy nowy punkt $(3 + 2/\sqrt{13}, 2 + 3/\sqrt{13})$.

Mamy $f\left(3 + \frac{2}{\sqrt{13}}, 2 + \frac{3}{\sqrt{13}}\right) = 15,61$ i jest to najwyższa wartość funkcji, którą możemy uzyskać zmieniając argument o wektor o długości 1.

Np. dla $[1,0]$ mamy $f(3 + 1, 2 + 0) = 8 + 6 = 14$

Wektor prostopadły do gradientu $[-3/\sqrt{13}, 2/\sqrt{13}]$ pokazuje kierunek, w którym wartość funkcji nie zmieni się.

Wektor przeciwny do gradientu $[-\frac{2}{\sqrt{13}}, -3/\sqrt{13}]$ pokazuje kierunek, w którym wartość funkcji najszybciej maleje.



Zielona i czerwona prosta to warstwyce funkcji $f(x, y) = 2x + 3y$

$$\nabla f(x, y) = [2, 3]$$

Algorytm propagacji wstecznej-jak działa

- Propagacja w przód (dane przechodzą przez wszystkie neurony sieci) wyznaczamy wynik sieci i stratę
- Propagacja wsteczna (obliczamy gradient funkcji kosztu i aktualizujemy wagi poprzez metodę spadku gradientowego).

*Gdy wszystkie dane uczące zostaną wykorzystane do aktualizacji parametrów sieci mówimy, że przetworzona została jedna **epoka**.*

Liczba epok, którą przyjmujemy, jest duża.

Dane, które przechodzą przez sieć, to batch. Nie zawsze wszystkie dane uczące są wykorzystywane do aktualizacji parametrów, mini-batch gradient descent.

Algorytm propagacji wstecznej

- **Krok do przodu: Propagacja w przód**

Wyznaczamy output $x^{(L)} = (x_1^{(L)}, \dots, x_{d^{(L)}}^{(L)})$ –wyniku sieci, neuronów ostatniej warstwy L ;

Czyli przechodzimy przez sieć do przodu, wyznaczając po drodze wyniki wszystkich neuronów. Oznaczamy przez $\hat{y}$ wynik sieci, wartość przewidzianą przez sieć.

Następnie obliczamy odległość wyniku sieci od prawdziwej wartości, czyli wyznaczamy funkcję straty. Jest ona funkcją wag i obciążeń neuronów sieci.

- **Propagacja wsteczna. Krok do tyłu**

Obliczenie gradientu funkcji kosztu i aktualizacja wag poprzez metodę spadku gradientowego.

Interesuje nas aktualizacja wag (wagi i obciążenie są zmiennymi funkcji kosztu).

Wektor

$$-\nabla C(w, b) = \begin{bmatrix} \vdots \\ 3 \\ \vdots \\ 0.1 \\ \vdots \end{bmatrix}$$

przeciwny do gradientu informuje, jak zmienić parametry modelu (wagi i obciążenia), by najefektywniej (najszybciej) zmniejszyć wartość funkcji kosztu. Algorytm propagacji wstecznej oblicza gradient. Wartości gradientu informują, jak wrażliwa jest funkcja straty na zmianę każdego parametru. Np. zmiana parametru, dla którego minus gradient wynosi 3, ma dużo większy wpływ na funkcję kosztu niż taka sama zmiana parametru, dla którego wartość przeciwna do gradientu wynosi 0.1.

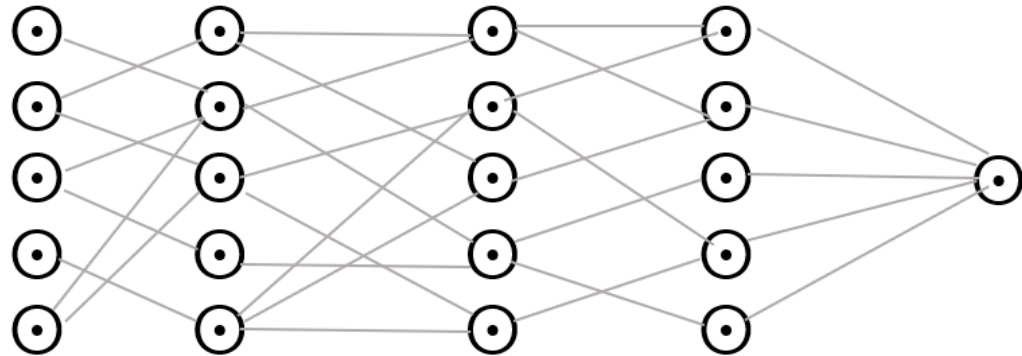
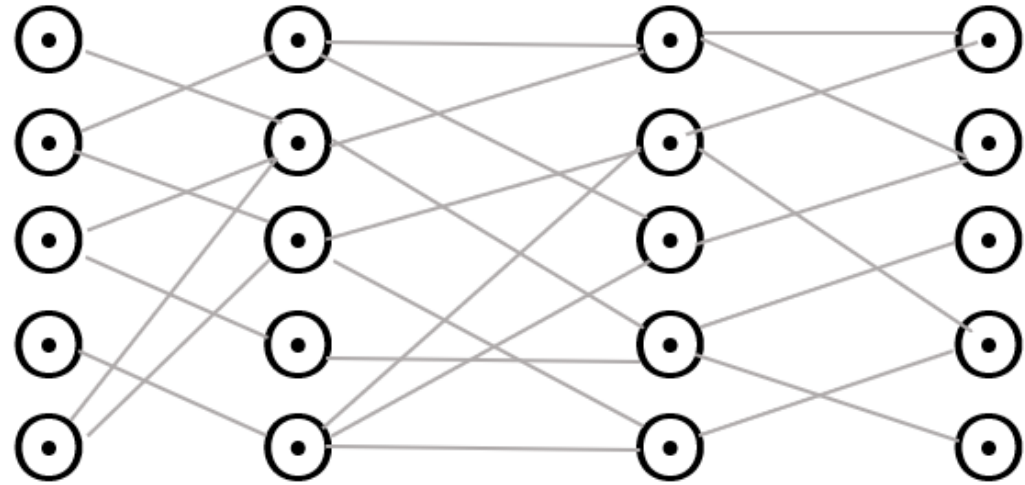
Obliczamy pochodne cząstkowe funkcji kosztu C względem wag i obciążeń. Wykorzystujemy regułę łańcuchową (liczenie pochodnych cząstkowych funkcji złożonych).

Gradient funkcji kosztu

$$\bullet -\nabla C(\underbrace{w, b}_{\text{wagi}}) = \begin{bmatrix} \vdots \\ 3 \\ \vdots \\ 0.1 \\ \vdots \end{bmatrix}$$

Wszystkie wagi modelu

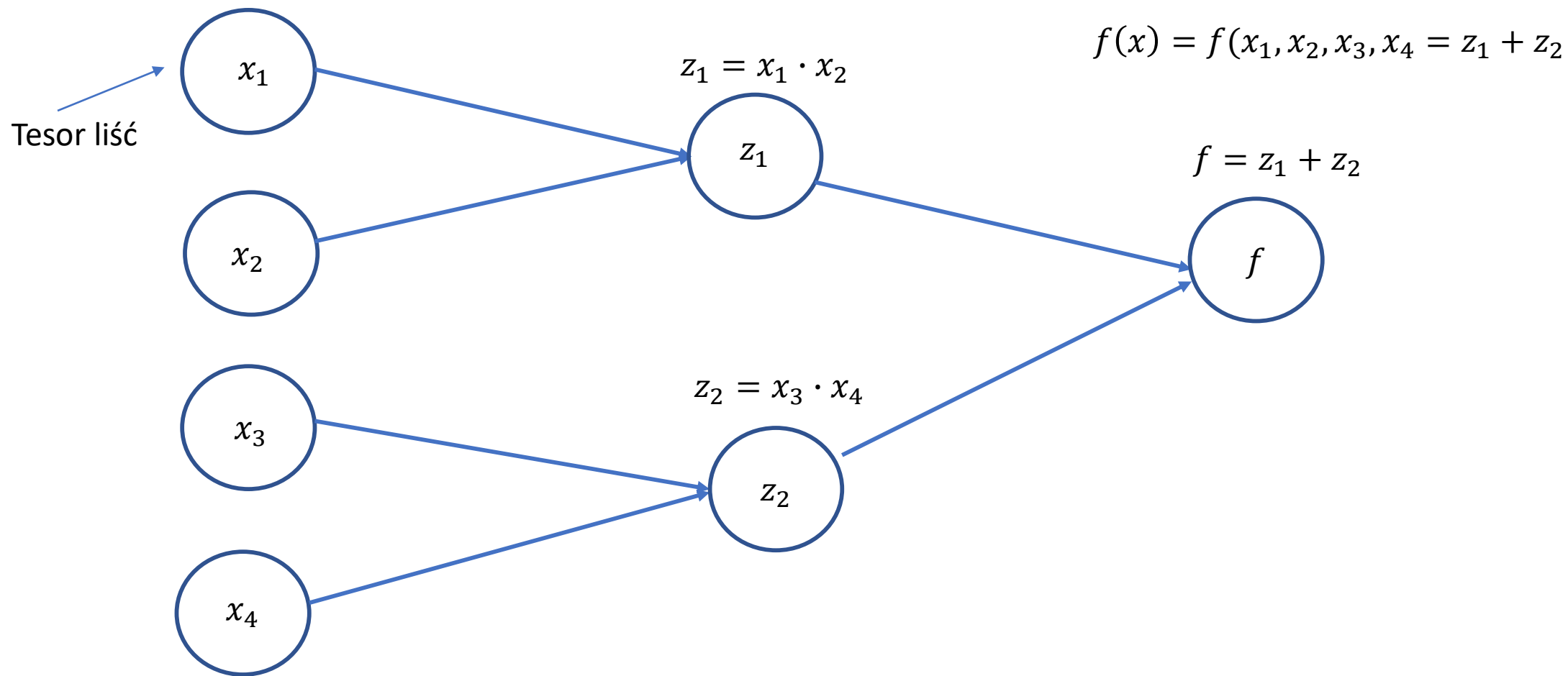
Gradient funkcji kosztu mówi nam, w jaki sposób zmienić wagi, aby spadek funkcji kosztów był najefektywniejszy, najszybszy.



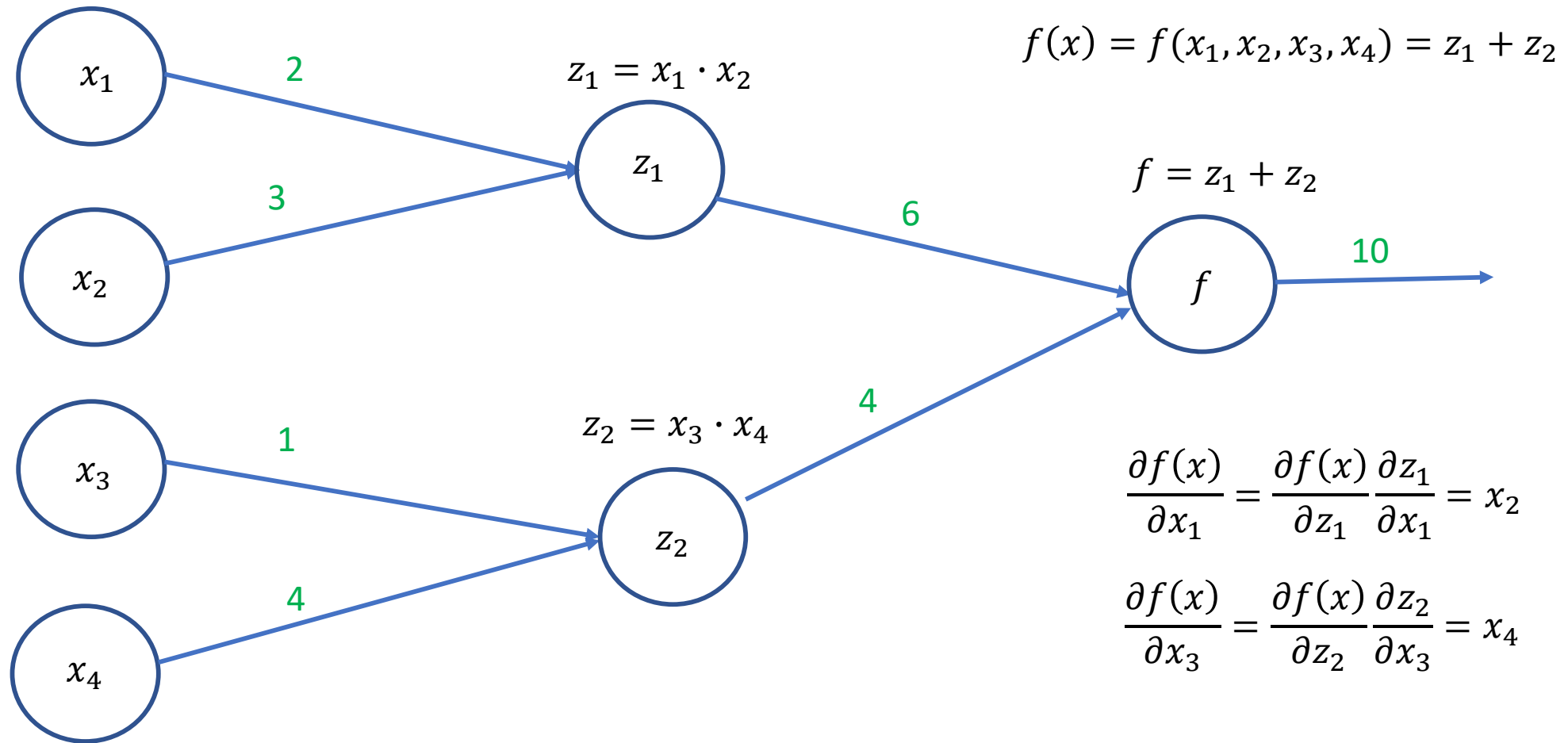
Praktyka: Grafy obliczeniowe w PyTorch

PyTorch wykorzystuje grafy obliczeniowe. Graf jest reprezentowany w PyTorch przez skierowany graf acykliczny (DAG), gdzie każdy węzeł oznacza obliczenie (np. dodawanie, mnożenie itp.), a każda krawędź oznacza przepływ danych między obliczeniami. Podczas definiowania modelu PyTorch, graf obliczeniowy jest tworzony poprzez zdefiniowanie przedniej funkcji modelu. Funkcja ta pobiera dane wejściowe i stosuje sekwencję operacji w celu uzyskania danych wyjściowych. Podczas przejścia do przodu PyTorch tworzy graf obliczeniowy, śledząc operacje oraz ich wejścia i wyjścia. Obliczenia modelu są reprezentowane przez graf, rodzaj struktury danych. Na etapie wstecznej propagacji graf jest wykorzystywany do obliczania gradientów i monitorowania zależności między obliczeniami.

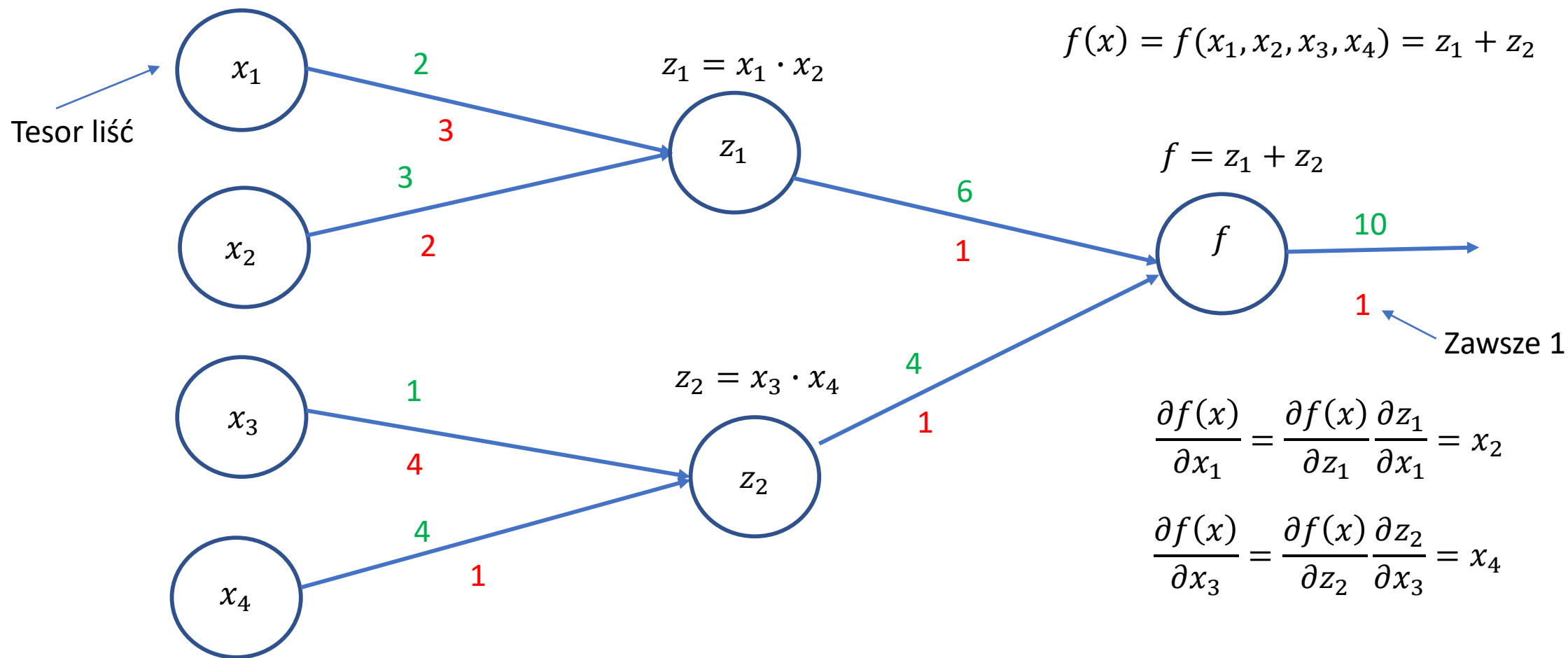
Przykład grafu obliczeniowego



Propagacja w przód (krok do przodu)



Propagacja wstecz



Wartości zielone: krok w przód
Wartości czerwone: krok wstecz

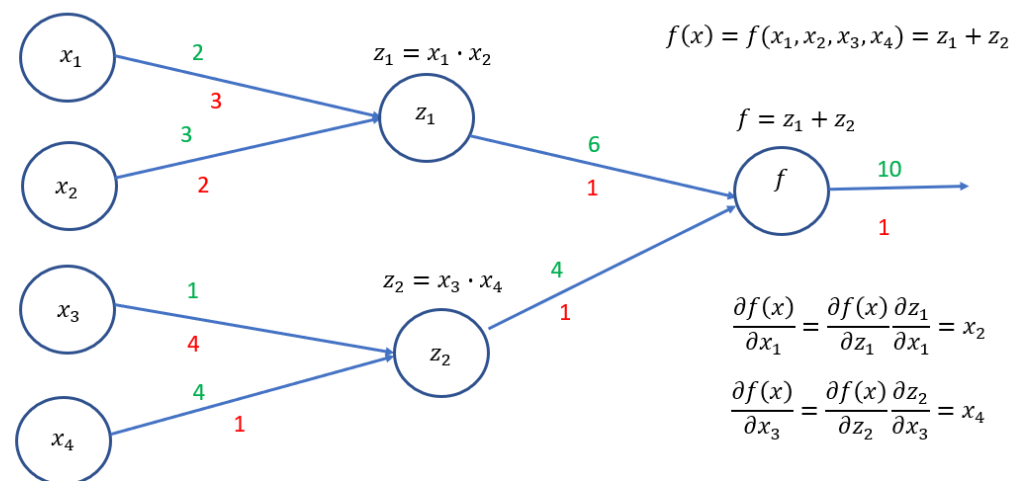
Autograd

```
x1=torch.tensor(2.,requires_grad=True,dtype=torch.float32)
x2=torch.tensor(3.,requires_grad=True,dtype=torch.float32)
x3=torch.tensor(1.,requires_grad=True,dtype=torch.float32)
x4=torch.tensor(4.,requires_grad=True, dtype=torch.float32)
```

```
z1=x1*x2
z2=x3*x4
f=z1+z2
df_dx=grad(outputs=f,inputs=[x1,x2,x3,x4])
```

```
print(f'gradient względem x1 = {df_dx[0]}')
print(f'gradient względem x2 = {df_dx[1]}')
print(f'gradient względem x3 = {df_dx[2]}')
print(f'gradient względem x4 = {df_dx[3]}')
```

```
gradient względem x1 = 3.0
gradient względem x2 = 2.0
gradient względem x3 = 4.0
gradient względem x4 = 1.0
```



Autograd w PyTorch

- <https://www.youtube.com/watch?v=MswxJw-8PvE>

Przykłady na ćwiczeniach

Powracamy do sieci i teorii

Oznaczenia:

l numer warstwy;

$l = L$ ostatnia warstwa, wynik, output (wynik sieci);

$l = 0$ pierwsza warstwa, input (dane);

$d^{(l)}$ – liczba neuronów w warstwie l , $d^{(L)}$ – liczba neuronów warstwy wyjścia;

Między warstwą $l - 1$ a l mamy $d^{(l-1)}d^{(l)}$ połączeń;

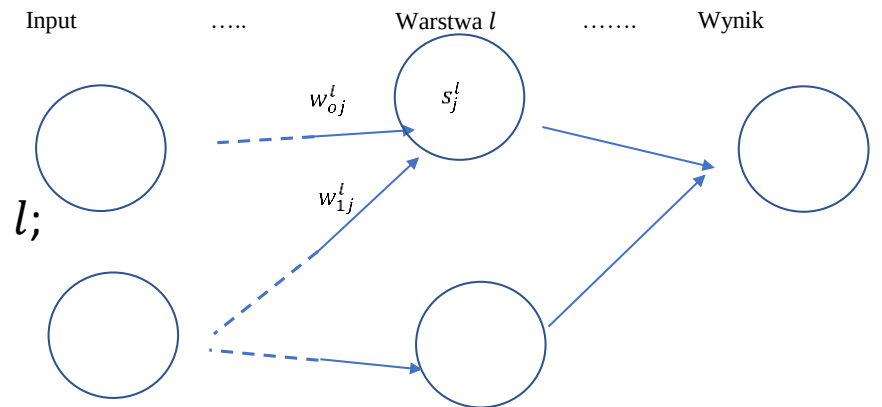
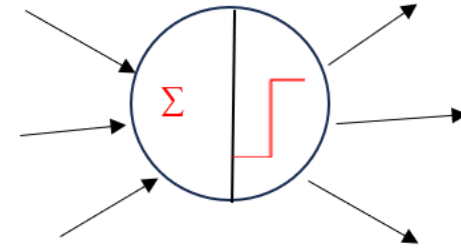
$w_{ij}^{(l)}$ – wagi dla $0 \leq l \leq L$; $0 \leq i \leq d^{(l-1)}$;

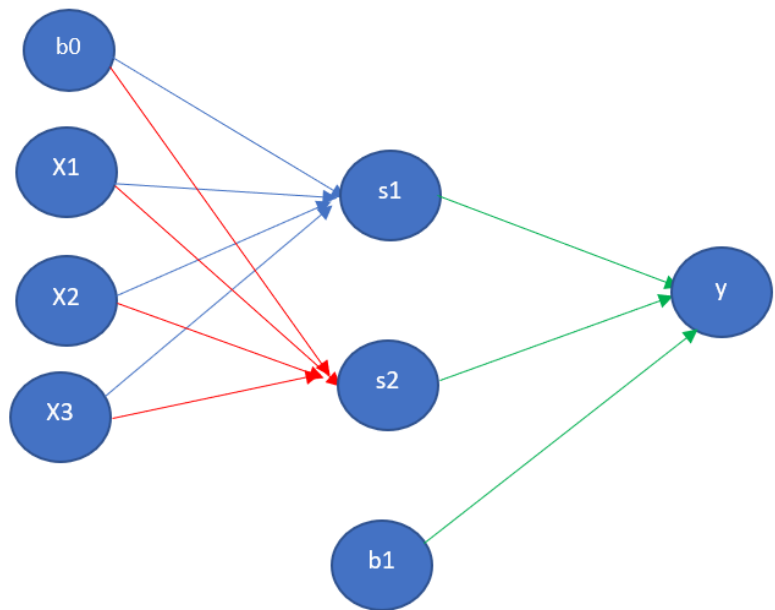
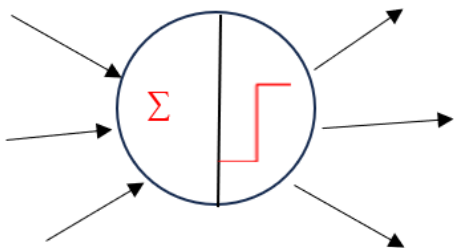
$w_{ij}^{(l)}$ – waga łącząca neuron i warstwy $l - 1$ z neuronem j warstwy l ;

$w_{0j}^{(l)} = b_j^l$ obciążenie, wymuszenie zewnętrzne

Input x_j^0 , $0 \leq j \leq d^{(0)}$

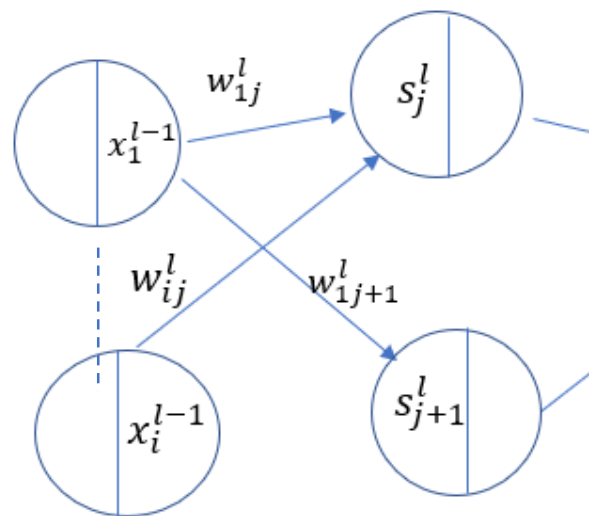
$x_j^{(l)}$ – wynik neuronu j warstwy l ; $x_j^{(L)}$ – wynik sieci, neuronów ostatniej warstwy;





To, co wchodzi do neuronu j l -tej warstwy

$$net_j^l = s_j^l = a_j^l = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} = \sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)}$$



To, co wychodzi z neuronu j l -tej warstwy

$$\begin{aligned} x_j^{(l)} &= \Phi(s_j^l) = \Phi(net_j) = \\ &= \Phi\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right) = \\ &= \phi\left(\sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)}\right). \end{aligned}$$

Neuron otrzymuje sygnał z neuronów warstwy poprzedniej. Sygnał ten nazywa się potencjałem wewnętrznym.

$$net_j^l = s_j^l = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} = \sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)}$$

Neuron przetwarza sygnał przy pomocy funkcji aktywacji/przejęcia Φ

$$x_j^{(l)} = \Phi(s_j) = \Phi(net_j) = \Phi\left(\sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)}\right) = \Phi\left(\sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)}\right).$$

Mamy: $C(w, b) = \frac{1}{2} \sum_{j=1}^{d^{(L)}} \left(x_j^{(L)} - y_j \right)^2 = \frac{1}{2} \|x^{(L)} - y\|^2$

$$x_j^{(L)} = \varphi(s_j^{(L)})$$

$$C(w, b) = \frac{1}{2} \sum_{j=1}^{d^{(L)}} \left(\varphi(s_j^{(L)}) - y_j \right)^2$$

Obliczamy pochodne cząstkowe wykorzystując regułę łańcucha:

$$\frac{\partial C(w, b)}{\partial w_{ij}^l} = \frac{\partial C(w, b)}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial w_{ij}^l}$$

$$net_j^l = s_j^l = \sum_{i=0}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} = \sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)}$$

Wprowadzamy oznaczenie: $\frac{\partial C(w,b)}{\partial s_j^{(l)}} = \delta_j^{(l)}$, pochodne te nazywamy deltami warstw i oznaczają wrażliwość funkcji kosztu względem sygnału $s_j^{(l)}$.

To co nas interesuje, to wyznaczenie $\delta_j^{(l)}$, które jest właśnie zadaniem algorytmu propagacji wstecznej.

Bezpośrednio ze wzoru na $s_j^{(l)}$ mamy: $\frac{\partial s_j^{(l)}}{\partial w_{ij}^l} = x_i^{(l-1)}$. Stąd

$$\frac{\partial C(w,b)}{\partial w_{ij}^l} = \frac{\partial C(w,b)}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial w_{ij}^l} = \delta_j^{(l)} x_i^{(l-1)}$$

Bez sumowania po j bo w_{ij} jest wagą między dwoma neuronami.

Obliczymy najpierw $\delta_j^{(L)}$, czyli deltę dla ostatniej warstwy:

$$C(w, b) = \frac{1}{2} \sum_{j=1}^{d^{(L)}} \left(\varphi(s_j^{(L)}) - y_j \right)^2$$

$$\delta_j^{(L)} = \frac{\partial C(w, b)}{\partial s_j^{(L)}} = \left(x_j^{(L)} - y_j \right) \varphi'(s_j^{(L)}).$$

Jeżeli φ jest funkcją sigmoidalną, czyli

$$\varphi(z) = \sigma(z) = \frac{1}{1+e^{-z}},$$

$$\text{to } \varphi'(z) = ((1 + e^{-z})^{-1})' = (-1)(-1)e^{-z}(1 + e^{-z})^{-2} =$$

$$(1 + e^{-z})^{-1}(1 - (1 + e^{-z})^{-1}) = \varphi(z)(1 - \varphi(z))$$

$$\delta_j^{(L)} = \frac{\partial C(w, b)}{\partial s_j^{(L)}} = \left(x_j^{(L)} - y_j \right) \sigma(s_j^{(L)})(1 - \sigma(s_j^{(L)})).$$

Obliczamy $\delta_i^{(l)}$ dla pozostałych warstw:

$$\delta_i^{(l)} = \frac{\partial C(w, b)}{\partial s_i^{(l)}}$$

$$\delta_i^{(l-1)} = \frac{\partial C(w, b)}{\partial s_i^{(l-1)}} = \sum_{j=1}^{d^{(l)}} \frac{\partial C(w, b)}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial s_i^{(l-1)}}$$

$$\frac{\partial s_j^{(l)}}{\partial s_i^{(l-1)}} = \frac{\partial}{\partial s_i^{(l-1)}} \left(\sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} x_i^{(l-1)} - b_j^{(l)} \right)$$

$$\frac{\partial}{\partial s_i^{(l-1)}} \left(\sum_{i=1}^{d^{(l-1)}} w_{ij}^{(l)} \Phi(s_i^{(l-1)}) - b_j^{(l)} \right) = w_{ij}^{(l)} \Phi'(s_i^{(l-1)})$$

$$\frac{\partial s_j^{(l)}}{\partial s_i^{(l-1)}} = w_{ij}^{(l)} \Phi'(s_i^{(l-1)})$$

$$\delta_i^{(l-1)} = \Phi'(s_i^{(l-1)}) \sum_{j=1}^{d^{(l)}} \delta_j^{(l)} w_{ij}^{(l)}$$

Powyższy wzór jest wzorem na δ algorytmu propagacji wstecznej.

Mamy także wzory dla obciążeń:

$$\frac{\partial \mathcal{C}(w, b)}{\partial b_j^{(l)}} = \frac{\partial \mathcal{C}(w, b)}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial b_j^{(l)}} = \delta_i^{(l)} (-1)$$

Algorytm spadku gradientowego dla sieci

η – stała uczenia

Dany układ początkowych wag i obciążeń (stałych)

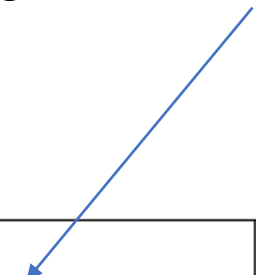
$w_{ij}^l(0), b_j^l(0)$. Wówczas

$$\begin{aligned}w_{ij}^{(\ell)}(n+1) &= w_{ij}^{(\ell)}(n) - \eta \delta_j^{(\ell)}(n) x_i^{(\ell-1)}(n) \\ b_j^{(\ell)}(n+1) &= b_j^{(\ell)}(n) + \eta \delta_j^{(\ell)}(n),\end{aligned}$$

$w_{ij}^l(n), b_j^l(n)$ wagi i obciążenie otrzymane w n -tym kroku

Algorytm propagacji wstecznej

Problem znikającego
gradientu



Cost formula	Deltas
$\frac{1}{2} \ x^{(L)} - z\ ^2$	$\delta_j^{(L)} = (x_j^{(L)} - z_j) \sigma'(s_j^{(L)})$
$-\sum z_k \ln x_k^{(L)}$	$\delta_j^{(L)} = -z_j (1 - \sigma(s_j^{(L)}))$
$-\sum z_k \ln x_k^{(L)} - (1 - z_k) \ln(1 - x_k^{(L)})$	$\delta_j^{(L)} = x_j^{(L)} - z_j$

σ – sigmoidalna funkcja aktywacji

Backpropagation-propagacja wsteczna

- Gdy wszystkie dane uczące zostaną wykorzystane do aktualizacji parametrów sieci mówimy, że przetworzona została jedna **epoka**.
- Proces aktualizacji parametrów trwa do momentu dopasowania systemu uczącego do danych treningowych.
- **Spadek gradientowy** jest stosowany do momentu, aż algorytm zbiegnie do lokalnie optymalnego rozwiązania.
- Jeśli model doskonale reprezentuje dane treningowe, to strata treningowa może być dowolnie mała.
- Jeśli model jest niedopasowany do danych treningowych, strata na zbiorze uczącym nie może spaść poniżej pewnego limitu.



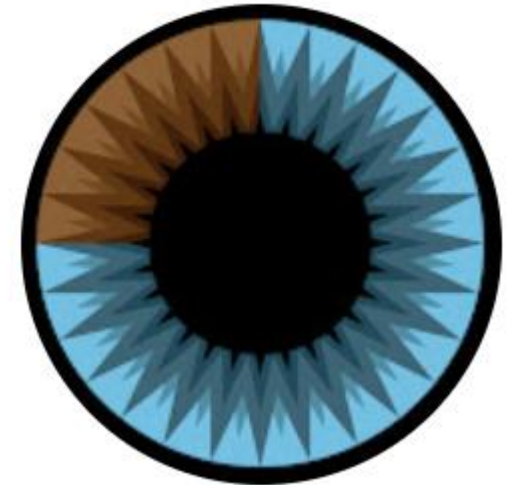
Grant Sanderson

@GrantSanderson · 108 tys. subskrybentów · 13 filmów

Sister channel to 3Blue1Brown for side projects. ...więcej

3blue1brown.com i jeszcze 2 linki

Subskrybuj



- <https://www.youtube.com/watch?v=llg3gGewQ5U>

Optymalizacja wag SGD

Procedura aktualizacji parametrów sieci przy użyciu metody gradientu jako średnia gradientów obliczanych na losowo wybranych próbkach uczących nazywana jest stochastyczną metodą spadku gradientowego (SGD)

Optymalizator Adam (adaptive moment estimation) to stochastyczna metoda spadku gradientowego, która opiera się na adaptacyjnej estymacji momentów pierwszego i drugiego rzędu.

Algorithms

Adadelta

Implements Adadelta algorithm.

Adafactor

Implements Adafactor algorithm.

Adagrad

Implements Adagrad algorithm.

Adam

Implements Adam algorithm.

AdamW

Implements AdamW algorithm.

LBFGS

Implements L-BFGS algorithm.

NAdam

Implements NAdam algorithm.

RAdam

Implements RAdam algorithm.

RMSprop

Implements RMSprop algorithm.

Rprop

Implements the resilient backpropagation algorithm.

SGD

Implements stochastic gradient descent (optionally with momentum).

Pakiety PyTorch

Pakiet	Opis
torch	Pakiet Torch zawiera struktury danych dla wielowymiarowych tensorów i definiuje operacje matematyczne na tych tensorach. Biblioteka tensorów.
torch.nn	Zawiera podstawowe moduły i klasy do budowy sieci: wstępnie zdefiniowane warstwy, funkcje aktywacji, funkcje strat i narzędzia do budowania i trenowania sieci neuronowych
torch.autograd	Pakiet do automatycznego różniczkowania
torch.nn.functional	Moduł oferuje zbiór funkcji do implementacjach sieci neuronowych (funkcje aktywacji, operacje splotu, funkcje kosztu).
torch.optim	Pakiet z optymalizatorami (SGD, Adam)
torch.utils	Zawiera klasy takie jak data loaders.
torchvision	Pakiet do przetwarzania obrazów: dane, modele, transformacje obrazów.

torch.nn Module	torch.nn.functional Module
It follows the Object-oriented approach with pre-defined layers as the Classes.	It is based on the Functional approach with stateless operations without any learnable operators.
It automatically manages parameters like weights, biases within layers	Since, the user has more control over the parameters, torch.nn.functional does not manage parameters automatically.
The Layers are integrated within the torch.nn.Module subclass. Thus, architecture becomes simple.	We have to use the Functions within custom functions/modules to implement the specific operations.